Codebase guide | Architecture map | Reviewer notes

Sarvam Developer Portal

Complete PDF guide to understand, navigate, explain, and modify the codebase. It covers routes, components, hooks, API flows, UI behavior, the custom diff algorithm, accessibility, errors, and engineering decisions.

Field	Value
Project	sarvam-intern-portal
Stack	Next.js 16, React 19, TypeScript, Tailwind CSS 4
Main routes	/playground, /diff-viewer, /documentation
Core requirements	Streaming inference playground and custom token-level diff view
Generated	17 May 2026

Use this PDF as your navigation map. If you want to change UI, start at the route page. If you want to change Sarvam calls, start in app/api. If you want to change algorithm behavior, start in app/utils/diffAlgorithm.ts.

1. Executive Summary

The Sarvam Developer Portal is a developer workflow app for testing browser-based inference and comparing model outputs. It is built as a Next.js App Router project with a fixed app shell, route-level feature pages, server-side API proxies, custom hooks for streaming and audio, and a custom token-level diff algorithm.

• The playground accepts text and audio input, streams model responses token by token, and displays live token, speed, and elapsed metrics.

• The diff viewer compares baseline and candidate model outputs side by side and highlights added and removed tokens.

• The API routes keep the Sarvam API key on the server and expose safe app-local endpoints for chat, comparison, and transcription.

• The UI uses a compact shadcn-like style with Sarvam-inspired gradient avatars, dark mode, toasts, and responsive layouts.

• The code intentionally avoids external diff libraries for the Part B algorithm requirement.

High-level request flow

```
User input -> React route -> custom hook -> Next.js API route -> Sarvam API -> stream or JSON -> UI state -> metrics/toasts/diff rendering
```

2. Project Structure

The app uses Next.js App Router. Product pages live under app, reusable shell components live in componets, and Sarvam API integration lives in app/api.

Path	Purpose	What it contains
app/layout.tsx	Root shell	Theme provider, toast provider, sidebar, metadata, desktop content offset.
app/page.tsx	Entry route	Redirects root URL to /playground.
app/globals.css	Global CSS	Tailwind import, dark variant, root colors, sr-only helper.
app/playground/page.tsx	Inference playground	Chat UI, markdown rendering, model selector, examples, persona, metrics, text/audio input, send/stop flow.
app/diff-viewer/page.tsx	Model diff view	Sample/live modes, prompt card, response cards, diff rendering, stats card, copy buttons.
app/documentation/page.tsx	In-app docs	Guide page with quick links, workflow steps, notes, and demo path.
app/hooks/useStream.ts	Streaming hook	Fetch + ReadableStream handling, SSE parsing, token metrics, abort/stop.
app/hooks/useAudioRecord.ts	Audio hook	getUserMedia, MediaRecorder, short-recording validation, transcription request.
app/utils/diffAlgorithm.ts	Diff engine	Tokenization, LCS dynamic programming, added/removed/unchanged output.
app/api/chat/route.ts	Streaming API	Server-side Sarvam chat proxy with stream: true.
app/api/compare/route.ts	Compare API	Parallel model calls with 15s timeout and JSON output.
app/api/transcribe/route.ts	Speech API	Forwards recorded audio to Sarvam speech-to-text.
componets/Sidebar.tsx	Navigation	Desktop sidebar, mobile header, docs link, dark toggle, dummy user.
componets/ToastProvider.tsx	Toast system	Context, toast variants, accessibility roles, dismiss behavior.
componets/ThemeProvider.tsx	Theme wrapper	Thin wrapper around next-themes.

3. Application Shell and Navigation

app/layout.tsx

This file is the global wrapper. It imports Inter, global CSS, Sidebar, ThemeProvider, and ToastProvider. Every route is rendered inside the same shell, which makes navigation, dark mode, and toast behavior consistent across pages.

```
<ThemeProvider defaultTheme="dark" enableSystem={false}>
<ToastProvider>
<Sidebar />
<main className="lg:pl-[256px]">{children}</main>
</ToastProvider>
</ThemeProvider>
```

Why this approach: providers are mounted once so every page can show toasts and read theme state. The desktop sidebar is fixed, so the main content uses matching left padding to prevent overlap.

componets/Sidebar.tsx

The sidebar is the navigation backbone. Desktop users get a fixed left sidebar. Mobile users get a top header with icon links. It contains Playground, Diff View, Documentation, dark mode, and a dummy engineer profile.

• usePathname highlights active navigation items.

• useTheme toggles light and dark mode.

• useSyncExternalStore is used as a hydration guard so theme UI does not mismatch.

• SarvamSquareAvatar gives the sidebar and profile a consistent gradient identity.

app/page.tsx and app/globals.css

• app/page.tsx redirects root traffic to /playground so the first screen is useful.

• globals.css imports Tailwind, defines dark mode variant, sets page colors, and defines .sr-only for screen-reader text.

4. Inference Playground

The playground is the largest route because it owns the live chat experience. It is organized into local helpers followed by the main Playground component.

Markdown response rendering

• parseMarkdownBlocks converts response text into headings, paragraphs, lists, quotes, code blocks, and horizontal rules.

• renderInlineMarkdown supports inline code, bold, italic, and links.

• AssistantMarkdown renders the parsed blocks and shows a blinking cursor while streaming.

Why this approach: model responses often contain Markdown. A lightweight custom renderer covers common cases without adding a heavy dependency, while keeping styling fully controlled.

Examples, model selector, and persona

• CHIPS contains the quick-start prompts shown on the empty playground screen.

• modelOptions defines the text model sarvam-m and voice model saaras:v3.

• ModelSelector lets users switch between text and voice modes from a compact dropdown.

• PersonaAvatar is the breathing gradient assistant identity; ThinkingPersona shows dots before tokens arrive and streamed text after they arrive.

Main playground state and flow

State or function	Role
inputMode	Tracks text or audio mode.
promptText	Current typed prompt or transcription result.
messages	Chat history rendered in the conversation.
elapsedMs	Drives elapsed and speed metrics while streaming.
sendPrompt	Adds user and assistant messages, clears input, starts stream.
handleComposerAction	Sends when idle, stops when streaming.
handleAudioMode	Starts or stops recording and prevents broken state switches.

Metrics

Metrics are derived from useStream and elapsed time. The mobile bar shows compact metrics at the top of the chat. The desktop right panel shows larger metric cards plus model, input mode, provider, and endpoint details.

• Tokens Generated: estimated by splitting streamed text chunks on whitespace.

• Speed: token count divided by elapsed seconds.

• Elapsed: updated every 100ms while streaming.

Input composer

• Textarea auto-resizes using useLayoutEffect to avoid flicker while typing.

• Enter sends the prompt; Shift + Enter adds a line break.

• The send button becomes a stop button while a response is streaming.

• The upload plus button is intentionally hidden and disabled because file upload is not supported.

5. Streaming and Audio Hooks

app/hooks/useStream.ts

useStream owns the browser-side streaming behavior. It posts messages to /api/chat, reads response.body with a ReadableStream reader, decodes chunks, parses SSE data, updates token metrics, and supports aborting.

Part	What it does
startStream	Posts prompt/messages and starts reading the stream.
response.body.getReader	Reads chunks as they arrive instead of waiting for full response.
TextDecoder	Converts byte chunks to UTF-8 strings.
parseSSEChunk	Extracts choices[0].delta.content from SSE data lines.
requestAnimationFrame	Batches UI updates for smoother streaming.
AbortController	Stops the stream and preserves partial output.

Why this approach: it exactly matches the assignment requirement for Fetch API and ReadableStream. It also lets the UI render live tokens and keep partial output on interruption.

app/hooks/useAudioRecord.ts

useAudioRecord owns microphone recording and transcription. It isolates browser MediaRecorder details from the playground UI.

• Requests mic access through navigator.mediaDevices.getUserMedia.

• Stores MediaRecorder and chunks in refs to avoid unnecessary rerenders.

• Rejects recordings shorter than 800ms or smaller than 900 bytes.

• Posts audio to /api/transcribe as FormData.

• Returns the transcript through onTranscriptionComplete.

• Shows toasts for short recordings, mic permission issues, no speech, and transcription failures.

Why this approach: fast stop/start recordings can produce empty blobs. The duration and byte-size guard prevents useless API calls and gives the user a clear message instead of a cryptic no transcript error.

6. API Routes

All Sarvam integration happens through app/api routes. This keeps the API key on the server and gives the frontend app-local endpoints.

app/api/chat/route.ts

• Uses Edge runtime for streaming-friendly behavior.

• Accepts prompt or messages.

• Normalizes only valid user, assistant, and system messages.

• Calls Sarvam chat completions with stream: true.

• Returns Sarvam response body directly as text/event-stream.

Why this approach: the server does not wait for full JSON. It pipes the stream directly so the client can display tokens as soon as Sarvam sends them.

app/api/compare/route.ts

• Accepts one prompt for both model versions.

• Runs two Sarvam calls in parallel with Promise.all.

• Uses different temperatures and system prompts to create baseline and candidate responses.

• Uses a 15 second AbortController timeout for each call.

• Returns modelA and modelB as JSON.

Why this approach: two sequential calls would stack latency. Parallel calls make comparison faster. Timeout handling prevents the UI from hanging when external API latency is high.

app/api/transcribe/route.ts

• Accepts audio FormData from the browser.

• Forwards the file to Sarvam speech-to-text with model saaras:v3.

• Returns transcript JSON to the frontend.

7. Model Output Diff View

The diff viewer is implemented in app/diff-viewer/page.tsx. It has sample mode for reliable demos and live mode for real API comparisons.

State and mode flow

State	Purpose
isLiveMode	Sample data vs real model comparison.
isGenerating	Locks UI while compare request is active.
livePrompt	Prompt used for live comparison.
modelAOutput / modelBOutput	Baseline and candidate text.
diffResult	Output from computeDiff, used for highlights and metrics.

Important components

• ModeToggle switches between Sample data and Use real model with gradient active states.

• PromptCard owns prompt draft state to prevent flicker while typing.

• PromptCard switches to live mode if the user clicks or types into the sample prompt preview.

• StatsCard shows total tokens, added, removed, unchanged, match percentage, and algorithm info.

• ModelCard displays each response with avatar, version badge, copy button, Diff/Text toggle, and fixed-height scroll body.

• HighlightedDiff maps DiffToken values into green additions, red removals, and normal unchanged tokens.

• getDiffStats calculates counts and percentages for summary metrics.

Why sample mode exists

Sample mode makes the diff viewer reliable even if the API is slow or unavailable. It also gives reviewers immediate examples of token changes, removals, and model update wording changes.

Why fixed response-card height matters

Model outputs can be long. If response cards grow with content, the layout jumps and the comparison becomes hard to scan. Fixed card height with internal scrolling keeps the two outputs aligned.

8. Custom Diff Algorithm

The diff algorithm lives in app/utils/diffAlgorithm.ts. It is custom code, not an external library, because the assignment requires building the core diff logic.

Algorithm steps

• Tokenize old and new text by splitting on whitespace while keeping whitespace tokens.

• Create a dynamic programming matrix for Longest Common Subsequence.

• Fill the matrix by comparing oldTokens[i - 1] and newTokens[j - 1].

• Backtrack from the bottom-right of the matrix.

• Mark diagonal matches as unchanged, upward moves as removed, and left moves as added.

• Return an ordered array of DiffToken objects.

```
type DiffType = 'added' | 'removed' | 'unchanged'
interface DiffToken { type: DiffType; value: string }

Time: O(n * m)
Space: O(n * m)
```

Why this approach

• Line-level diff is too coarse for model outputs because a single sentence can change by only one or two words.

• Token-level LCS is deterministic and easy to explain in a review.

• Preserving whitespace keeps reconstructed output readable after highlighting.

• Myers diff is more efficient for large diffs but more complex to implement and explain. For short to medium model responses, LCS is a good assignment fit.

Limitations

• It compares exact tokens, not semantic meaning.

• Punctuation may stay attached to nearby words depending on tokenization.

• The O(n * m) matrix is not ideal for very large documents.

• The match percentage reflects token overlap, not model quality.

9. UI, Responsiveness, and Visual System

• The UI uses compact cards, low-contrast borders, dark surfaces, and restrained typography.

• Sarvam-style gradient avatars and chips create brand continuity across sidebar, models, persona, examples, and docs.

• Grain overlays reduce overly shiny gradients and make them feel richer.

• Desktop layout uses fixed sidebar plus optional right metrics panel.

• Mobile layout uses top navigation, compact metrics, stacked cards, and tiny two-column examples.

Documentation page

app/documentation/page.tsx is a lightweight in-app guide. It explains how to use the playground and diff view, gives quick links, and provides a recommended demo path.

ToastProvider

The custom toast system provides minimal feedback without bringing in a UI package. It supports default, success, warning, and destructive variants, and uses role=status or role=alert for accessibility.

10. Accessibility and Error Handling

Accessibility

• Chat conversation has aria-live=polite, aria-label, and aria-atomic=false.

• Thinking and streaming states include sr-only announcements.

• Icon-only buttons have aria-labels.

• Toast notifications use live regions and alert/status roles.

• Keyboard flow supports Enter to send and Shift + Enter for new lines.

• prefers-reduced-motion is respected by global animation CSS in the playground.

Error handling

Area	Issue	Handling
Stream	Network drop or interruption	Partial output preserved and Stream interrupted toast shown.
Stream	User stops response	AbortController stops stream and Stream stopped toast appears.
Audio	Recording too short	Duration and byte-size guard with warning toast.
Audio	Mic blocked	audioError plus destructive toast.
Transcription	No transcript or API failure	User-facing message and toast.
Compare	External API slow	15 second timeout returns clean 504 and UI toast.
Clipboard	Copy fails	Destructive toast with fallback guidance.

11. How to Modify the Code Quickly

Task	Where to edit
Change playground examples	app/playground/page.tsx -> CHIPS
Change diff examples	app/diff-viewer/page.tsx -> mockScenarios
Change chat streaming	app/hooks/useStream.ts and app/api/chat/route.ts
Change audio recording	app/hooks/useAudioRecord.ts and app/api/transcribe/route.ts
Change compare prompts or timeout	app/api/compare/route.ts
Change diff algorithm	app/utils/diffAlgorithm.ts
Change response cards	app/diff-viewer/page.tsx -> ModelCard and HighlightedDiff
Change metrics	app/playground/page.tsx -> RightSidebar, CompactMetricsBar; app/diff-viewer/page.tsx -> StatsCard
Change toasts	componets/ToastProvider.tsx
Change sidebar/profile/docs link	componets/Sidebar.tsx
Change docs content	app/documentation/page.tsx
Change dark mode default	app/layout.tsx -> ThemeProvider props

Mental model

Route-specific behavior lives in the route page. Shared shell behavior lives in componets. Sarvam calls live in app/api. Algorithm behavior lives in app/utils. Browser-side network or media behavior lives in app/hooks.

12. Running, Deployment, and Final Review

Commands

```
npm install
npm run dev
npm run lint
npx tsc --noEmit
npm run build
```

Environment

```
SARVAM_API_KEY=your_sarvam_api_key_here
```

Deployment checklist

• Use Node.js 20.9 or newer.

• Add SARVAM_API_KEY in Vercel or Netlify environment variables.

• Push the final repository to GitHub.

• Deploy the app and verify /playground, /diff-viewer, and /documentation.

• Record the 3-minute walkthrough on the deployed app if possible.

• Add GitHub, deployed app, and video links to the final PDF submission document.

Demo script

• Open Playground and click an example prompt.

• Show tokens, speed, and elapsed metrics updating while response streams.

- Stop one stream and point out that partial output is preserved.
- Switch to voice mode and explain the transcription path.
- Open Diff View and click sample examples.
- Switch to live mode, generate a comparison, and show token-level highlights.
- Point to the algorithm metric card and explain LCS with O(n * m) complexity.

13. Known Limitations and Future Improvements

- Chat stream handles interruption and manual stop, but an explicit client-side timeout for chat stalls could be added.
- The custom model selector can be improved with full arrow-key navigation.
- Token counting is approximate because it uses whitespace splitting instead of a model tokenizer.
- The LCS diff is not optimized for very large texts.
- The diff view compares exact token changes, not semantic equivalence.
- A future fleet configuration/deployment UI would complete the third workflow mentioned in the assignment situation.
- Automated unit tests could be added for parseSSEChunk, computeDiff, and API error paths.